

Architektur ist überbewertet – und was wir daraus lernen können

11.12.2024 11:30 Uhr Golo Roden



(Bild: Erstellt mit KI (Midjourney) durch iX)

Architektur wirkt oft wie eine dunkle Magie, die nur in den elitären Kreisen der Wissenden diskutiert wird. Das ist nicht nur falsch, sondern auch gefährlich.

"Architektur ist überbewertet" wirkt auf den ersten Blick wie ein typischer Clickbait-Titel, aber ich will kurz erklären, was genau ich mit diesem Titel meine. Architektur hat ein Problem, über das meiner Meinung nach viel zu wenig gesprochen wird, das aber auf uns als Entwicklerinnen und Entwickler gravierende Auswirkungen hat.

THE NEXT BIG THING – GOLO RODEN

Es geht um die ständige Idealisierung und die überhöhte Wahrnehmung von Architektur, von Architekten und all dem, was damit zusammenhängt und einhergeht, wie zum Beispiel einer ganzen Reihe von Prinzipien, Konzepten und nicht zuletzt auch von einigen Personen.

Das schreckt ab, denn auf dem Weg wird Architektur nicht mehr als das wahrgenommen, was sie eigentlich ist – nämlich als die Kunst, Software adäquat zu strukturieren –, sondern sie wird auf

einmal als Selbstzweck wahrgenommen. Das jedoch ist nicht nur falsch, sondern auch gefährlich.

Eine emotionale und keine sachliche Diskussion

Architektur hat ein Problem: Sie wird idealisiert und überhöht. Wenn Sie nicht genau wissen, was ich damit meine, stellen Sie sich vor, wie über typische Prinzipien und Konzepte aus der Architektur gesprochen wird: Da geht es nicht um ein paar hilfreiche Leitplanken und nützliche Wegweiser, die Sie bei Ihren Bemühungen zu einer gut strukturierten Software unterstützen. Nein, da geht es zum Beispiel um Clean Code (und ja, mir ist bewusst, dass das Thema für viele nicht unter Architektur fällt, im Sinne einer besseren Strukturierung von Software gehört aber auch Clean Code dazu).

Allein durch die Namensgebung sprechen wir nicht mehr nur auf einer technischen Ebene, sondern es schwingt noch etwas ganz anderes mit, etwas Emotionalisiertes: Wenn Sie sich nicht an diese Regeln halten, dann schreiben Sie Code, der schmutzig, ungepflegt und minderwertig ist. Wollen Sie so etwas etwa wirklich? Wo die Frage nach gut strukturiertem Code explizit auf der emotionalen Ebene exkludiert, fällt eine rein sachliche Diskussion schwierig.

Ein weiteres Beispiel sind die SOLID-Prinzipien. Auch suggeriert allein der Name bereits ein stabiles Fundament, ein Fels in der Brandung. Und wehe, Sie setzen sie nicht ein: Dann stehen Sie da wie ein Grashalm, der beim leichtesten Windhauch umknickt. Wollen Sie das?

Bei diesen Beispielen kann man sich sehr gut den erhobenen Zeigefinger vorstellen: Passen Sie bloß auf, sonst enden Sie wie all die anderen vor Ihnen – als Versager.

Fanatischer Eifer und Personenkult

Tatsächlich ist es jedoch nicht die Namensgebung allein: Das ist nur das, was als Erstes auffällt. Es geht noch weiter: Versuchen Sie einmal, Clean Code zu kritisieren. Sofort kommen zahlreiche Befürworterinnen und Befürworter, die Clean Code mit Leidenschaft verteidigen und als das Nonplusultra darstellen. So nach dem Motto:

"Wie können Sie es wagen, Clean Code zu hinterfragen, zu kritisieren oder gar infrage zu stellen? Was bildet Sie sich ein, wer Sie sind? Und übrigens: Sie haben damit auch Uncle Bob beleidigt, also entschuldigen Sie sich gefälligst!"

Vielleicht finden Sie das jetzt übertrieben, und natürlich verhält sich nicht jede Anhängerin und jeder Anhänger von Clean Code, SOLID & Co. derart. Das Problem ist jedoch: Es lässt sich nicht leugnen, dass um diese Themen ein gewisser Kult betrieben wird, inklusive eines ausgeprägten Personenkults. Ich behaupte nicht, dass das nur Uncle Bob alias Robert C. Martin betrifft, aber bei ihm ist es besonders auffällig. Und so etwas ist immer problematisch, denn dann wird nicht mehr sachlich über die Inhalte diskutiert. Stattdessen werden diese mit unsachlichen Emotionen und persönlichen Befindlichkeiten aufgeladen und untrennbar verknüpft.

Architektur? Ein Thema für die Elite

Wenn sich das über Jahre und Jahrzehnte verfestigt, hat das natürlich Folgen. Architektur und lesbarer Code werden dann nicht mehr als etwas wahrgenommen, womit sich jede und jeder beschäftigen sollte, sondern als etwas für die Eliten. Genau so erlebe ich oft die Meinung von weniger erfahrenen Entwicklerinnen und Entwicklern: Sie sagen, sie hätten von Architektur keine Ahnung, aber eines Tages würden sie das gerne in ihrem Jobtitel stehen haben, "Distinguished Senior Solution Architect" oder etwas Ähnliches.

So entsteht eine mystische Aura um dieses Thema, als wäre es nur für sehr erfahrene Menschen zugänglich, die es geschafft haben, Teil dieses elitären Kreises zu werden. Was genau eine Architektin oder ein Architekt eigentlich macht, wird dabei oft nicht mehr klar. Es scheint nur eines sicher: Es handelt sich um eine intellektuell anspruchsvolle Aufgabe, der Normalsterbliche vermeintlich nicht gewachsen sind. Dementsprechend beschäftigt man sich nicht mit Architektur, überlässt es den Weisen, entwickelt sich nicht weiter und bleibt jahrelang in dem Traum gefangen, selbst irgendwann ein Teil dieses elitären Zirkels zu sein, ohne den Weg dorthin zu kennen.

Kein Selbstzweck

Das eigentliche Problem ist, dass Architektur oft als Selbstzweck betrachtet wird: Sie wird um ihrer selbst willen betrieben und nicht als praktisches Werkzeug, das einer größeren Aufgabe dient. Nein, die Architektur selbst wird zur höheren Aufgabe erhoben. Das ist jedoch Unsinn. Clean Code kann durchaus kritisiert werden, und Robert C. Martin ist kein Heiliger. Die SOLID-Prinzipien sind, wenn wir ehrlich sind, fünf relativ beliebig ausgewählte Prinzipien, die in der Objektorientierung mehr oder weniger sinnvoll und manchmal nützlich sind.

Warum es ausgerechnet diese fünf Prinzipien geworden sind und nicht irgendwelche anderen, bleibt Spekulation. Vermutlich hat sich jemand ein schickes Akronym ausgedacht (eben "SOLID") und dazu Prinzipien zusammengesucht, die ansatzweise sinnvoll erschienen. Wenn man sich diese Prinzipien genauer anschaut, sind die meisten davon weder besonders elegant formuliert (im Sinne von anschaulich und greifbar), noch sind sie besonders hilfreich im Alltag. Ich kann mich zumindest nicht erinnern, wann ich das letzte Mal dachte:

"Wie praktisch, dass ich das Liskovsche Substitutionsprinzip kenne, ohne das wäre diese Struktur deutlich schlechter."

Dennoch wird vermittelt, dass die SOLID-Prinzipien die fünf wichtigsten Prinzipien der objektorientierten Programmierung seien. Das sind sie jedoch nicht, da die meisten viel zu spezifisch sind.

Das heißt, sie sind keine absolute Wahrheit, die nicht hinterfragt werden dürfte. Sie sind nur Mittel zum Zweck. Architektur (und dazu zähle ich Clean Code und diese Prinzipien wie oben schon erwähnt) hat letztlich nämlich nur ein einziges konkretes Ziel: Sie soll helfen, eine gute Struktur für Code zu schaffen, sodass dieser langfristig verständlich, überschaubar, wartbar und erweiterbar bleibt.

Der eigentliche Sinn von Architektur

Wann hat Code diese Struktur? Eigentlich ist das ganz einfach: immer dann, wenn zwei Grundprinzipien erfüllt sind. Code ist gut strukturiert, wenn er zum einen eine geringe Kopplung und zum anderen eine hohe Kohäsion aufweist. In dem **Video zu Kopplung und Kohäsion [2]** erkläre ich die beiden Prinzipien im Detail.

Diese beiden Prinzipien – geringe Kopplung und hohe Kohäsion – sind der Leitstern, der uns überall im Code Orientierung gibt, egal ob bei Funktionen und Klassen im Kleinen oder im Großen bei den zahlreichen Microservices einer verteilten Anwendung. Diese Prinzipien machen eine gute Architektur aus. Darüber nachzudenken und daran zu arbeiten erfordert keinen Titel wie "Distinguished Senior Solution Architect". Auch Praktikantinnen und Praktikanten können sich damit beschäftigen. Vielleicht fehlt ihnen etwas Erfahrung, aber sie sind durchaus intellektuell in der Lage, darüber nachzudenken.

Die entscheidende Frage lautet: Wo sollte ich mich um was kümmern, wer ist wofür verantwortlich? Das ist letztlich alles.

Die Entscheidungen müssen ein höheres Ziel unterstützen: die Fachlichkeit. Denn genau darum geht es in der Softwareentwicklung. Wir entwickeln nicht deshalb Software, weil es Spaß macht (auch wenn es tatsächlich Spaß macht), sondern um ein tieferliegendes fachliches Problem zu lösen. Software ist kein Selbstzweck, sondern ein Werkzeug, um einen fachlichen Zweck zu erfüllen. Und Architektur muss sich daran messen lassen.

Es geht nicht darum, ob wir alle fünf SOLID-Prinzipien umgesetzt oder möglichst viele Design-Patterns verwendet haben. Was am Ende zählt, ist, dass die Architektur das fachliche Problem unterstützt und es uns erleichtert, die Software langfristig zu warten und weiterzuentwickeln. Apropos Design-Patterns: **Auch diese halte ich für überbewertet [3].**

Falsche, weil zu frühe Entscheidungen

Mit diesem Mindset trifft man jedoch oft auf Projekte, in denen die Architektur schon feststand, bevor die fachlichen Anforderungen bekannt waren. Das halte ich für eine schlechte Idee, genauso wie die Wahl von Technologien, bevor Architektur und Anforderungen klar sind. Was ich in Teams oft erlebe, sind Argumente wie:

"Wir sind aber nicht Netflix. Wir brauchen keine Microservices, kein CQRS, kein Event-Sourcing, keine komplexe Architektur."

Stattdessen setzt man auf eine einfache Drei-Schichten-Architektur mit einem Monolithen und Dependency Injection. Missverstehen Sie mich nicht: Es gibt Projekte, wo das angemessen ist. Viel zu oft werden solche Entscheidungen jedoch mit unpassenden Argumenten und aus den falschen Gründen getroffen. Bei Microservices etwa ist die Frage nicht:

"Sind Sie Netflix?"

Stattdessen geht es (eigentlich) um geringe Kopplung und hohe Kohäsion bei ausreichend komplexer Fachlichkeit. Diese Anforderungen kann auch ein Drei-Personen-Start-up haben. Das hängt nicht davon ab, ob man Netflix ist, sondern vom jeweiligen Business.

Nachteile von "einfacher" Architektur

Die klassische Drei-Schichten-Architektur im Monolithen hat einige gravierende Nachteile: Fachlicher und technischer Code sind oft nicht sauber getrennt, was Anpassungen erschwert. Beim Testen merkt man das besonders: Es ist kaum möglich, die Geschäftslogik zu testen, weil immer die Datenbank mit dranhängt. Statt die Architektur zu hinterfragen, setzen viele dann auf Mocking. Doch Mocking ist eine der schlechtesten Ideen im Testen, weil es grüne Tests liefert, ohne sicherzustellen, dass die Mocks die Realität abbilden. Mit anderen Worten: Mocking wird zum Workaround für schlechte Architekturentscheidungen.

Tatsache ist: Es gibt deutlich bessere Ansätze. Gerade bei komplexer Geschäftslogik bietet sich etwa eine hexagonale Architektur an, die die Fachlichkeit in den Mittelpunkt rückt – ohne Abhängigkeiten zu externen Ressourcen. Doch auch hier heißt es oft:

"Wir sind doch nicht Netflix."

Solche Aussagen basieren oft auf Angst vor Veränderung. Das Problem sind nicht sachliche Argumente, sondern emotionale Bedenken:

"Das haben wir noch nie gemacht, also bleibt alles beim Alten."

Oft bekommen dann Berater den Auftrag, die beruhigende Botschaft zu verbreiten, ein Monolith reiche aus, denn man sei nicht Netflix. Das wird dann nach und nach zum gefährlichen Mantra.

Jahre später stellt man dann fest, dass die Technologie veraltet ist, aber ein Wechsel kaum möglich ist, weil alles miteinander verwoben ist. Technologiewechsel werden so zu Alles-oder-Nichts-Entscheidungen. Dabei treten viele Probleme auf: schlechte Testbarkeit, keine Skalierbarkeit, fehlende APIs für externe Systeme oder unklare Verantwortlichkeiten bei Master-Data-Management. All das hätte man vermeiden können, wenn man früher über eine fachlich sinnvolle Architektur nachgedacht hätte.

Was bedeutet das alles?

Der Sinn einer Architektur liegt darin, eine tragfähige Struktur für die Software zu schaffen, die geringe Kopplung und hohe Kohäsion gewährleistet. Wie man das erreicht, ist zweitrangig. Wichtig ist, dass die Fachlichkeit unterstützt wird. Unsachliche Diskussionen über Werkzeuge wie Clean Code, SOLID oder Design-Patterns helfen dabei nicht. Stattdessen sollten wir mehr darüber sprechen, warum wir Software entwickeln und wie die Fachlichkeit unsere Entscheidungen

beeinflusst. Wer sich weiterhin hinter emotionalen und unsachlichen Argumenten versteckt, wird langfristig viel Schaden anrichten.

Fragen Sie sich daher immer, ob Ihre Architektur das fachliche Problem löst oder eher behindert. Und wenn Letzteres der Fall ist, denken Sie über Alternativen nach – losgelöst von dem, was in Lehrbüchern steht oder als Standard gilt. Architektur ist mehr als das. Jede Entwicklerin und jeder Entwickler kann dazu beitragen, denn Programmieren bedeutet, Strukturen zu schaffen.

IX-SONDERHEFT SOFTWAREARCHITEKTUR

(rme [5])

URL dieses Artikels:

<https://www.heise.de/-10191624>

Links in diesem Artikel:

[1] <https://www.heise.de/Datenschutzerklaerung-der-Heise-Medien-GmbH-Co-KG-4860.html>

[2] <https://www.youtube.com/watch?v=kWP432pXP-U>

[3] <https://www.youtube.com/watch?v=oC-9HZ0r3e4>

[4] [https://shop.heise.de/ix-developer-praxis-softwarearchitektur-2024/print?
wt_mc=intern.shop.shop.ix-dev-softwarearchitektur24.dos.textlink.textlink](https://shop.heise.de/ix-developer-praxis-softwarearchitektur-2024/print?wt_mc=intern.shop.shop.ix-dev-softwarearchitektur24.dos.textlink.textlink)

[5] <mailto:rme@ix.de>

Copyright © 2024 Heise Medien